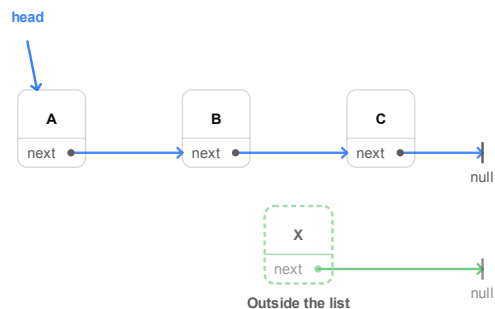


Lab Exercise: Singly Linked List Operations

Aim

Write a **C** program to implement a singly linked list and perform the following operations:

1. Insertion at the beginning
2. Insertion at the end
3. Insertion at a specified position
4. Deletion from the beginning
5. Deletion from the end
6. Deletion from a specified position
7. Traversal/display of the linked list



X → null

Step 1 of 4

Insert X after B: Initialize X.

Insert

Delete

Step

1

[Give feedback](#)

Algorithm

Insertion at beginning

1. Create a new node.
2. Store the value in the node.
3. Set `newNode->next = head`.
4. Set `head = newNode`.

Insertion at end

1. Create a new node.
2. If the list is empty, make it the `head`.
3. Otherwise, traverse to the last node.
4. Set the last node's `next` to the new node.

Deletion from beginning

1. Check whether the list is empty.
2. Store `head` in a temporary pointer.
3. Move `head` to `head->next`.
4. Free the temporary node.

Deletion from end

1. Check whether the list is empty.
2. Traverse to the last node while keeping track of the previous node.
3. Set the previous node's `next` to `NULL`.
4. Free the last node.

Traversal

1. Start from `head`.
2. Visit each node.
3. Print its data.
4. Move to the next node until `NULL`.

C Program: Menu-Driven Singly Linked List

C

```
#include <stdio.h>
#include <stdlib.h>

// Structure of a node
struct Node {
    int data;
    struct Node *next;
};

// Function to create a new node
struct Node* createNode(int value) {
    struct Node *newNode;
```

```

newNode = (struct Node*)malloc(sizeof(struct Node));

if (newNode == NULL) {
    printf("Memory allocation failed!\n");
    exit(1);
}

newNode->data = value;
newNode->next = NULL;

return newNode;
}

// Insert at beginning
void insertBeginning(struct Node **head, int value) {
    struct Node *newNode = createNode(value);

    newNode->next = *head;
    *head = newNode;

    printf("%d inserted at beginning.\n", value);
}

// Insert at end
void insertEnd(struct Node **head, int value) {
    struct Node *newNode = createNode(value);
    struct Node *temp;

    if (*head == NULL) {
        *head = newNode;
        printf("%d inserted at end.\n", value);
        return;
    }

    temp = *head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;

    printf("%d inserted at end.\n", value);
}

// Insert at a specified position
void insertPosition(struct Node **head, int value, int position) {
    struct Node *newNode;
    struct Node *temp;
    int i;

```

```

    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }

    // Position 1 means insertion at beginning
    if (position == 1) {
        insertBeginning(head, value);
        return;
    }

    if (*head == NULL) {
        printf("Invalid position! List is empty.\n");
        return;
    }

    temp = *head;

    for (i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position!\n");
        return;
    }

    newNode = createNode(value);

    newNode->next = temp->next;
    temp->next = newNode;

    printf("%d inserted at position %d.\n", value, position);
}

// Delete from beginning
void deleteBeginning(struct Node **head) {
    struct Node *temp;

    if (*head == NULL) {
        printf("List is empty. Deletion not possible.\n");
        return;
    }

    temp = *head;
    *head = (*head)->next;

    printf("%d deleted from beginning.\n", temp->data);

    free(temp);
}

```

```

// Delete from end
void deleteEnd(struct Node **head) {
    struct Node *temp;
    struct Node *previous;

    if (*head == NULL) {
        printf("List is empty. Deletion not possible.\n");
        return;
    }

    // Only one node
    if ((*head)->next == NULL) {
        printf("%d deleted from end.\n", (*head)->data);
        free(*head);
        *head = NULL;
        return;
    }

    temp = *head;
    previous = NULL;

    while (temp->next != NULL) {
        previous = temp;
        temp = temp->next;
    }

    previous->next = NULL;

    printf("%d deleted from end.\n", temp->data);

    free(temp);
}

// Delete from a specified position
void deletePosition(struct Node **head, int position) {
    struct Node *temp;
    struct Node *previous;
    int i;

    if (*head == NULL) {
        printf("List is empty. Deletion not possible.\n");
        return;
    }

    if (position < 1) {
        printf("Invalid position!\n");
        return;
    }

    // Position 1 means deletion from beginning
    if (position == 1) {
        deleteBeginning(head);
    }
}

```

```

        return;
    }

    temp = *head;
    previous = NULL;

    for (i = 1; i < position && temp != NULL; i++) {
        previous = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Invalid position!\n");
        return;
    }

    previous->next = temp->next;

    printf("%d deleted from position %d.\n", temp->data, position);

    free(temp);
}

// Traverse and display the list
void display(struct Node *head) {
    struct Node *temp;

    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    printf("Linked List: ");

    while (temp != NULL) {
        printf("%d", temp->data);

        if (temp->next != NULL) {
            printf(" -> ");
        }

        temp = temp->next;
    }

    printf(" -> NULL\n");
}

// Main function
int main() {
    struct Node *head = NULL;

```

```

int choice;
int value;
int position;

do {
    printf("\n===== SINGLY LINKED LIST =====\n");
    printf("1. Insert at Beginning\n");
    printf("2. Insert at End\n");
    printf("3. Insert at Position\n");
    printf("4. Delete from Beginning\n");
    printf("5. Delete from End\n");
    printf("6. Delete from Position\n");
    printf("7. Display / Traversal\n");
    printf("8. Exit\n");
    printf("=====\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {

        case 1:
            printf("Enter value: ");
            scanf("%d", &value);

            insertBeginning(&head, value);
            break;

        case 2:
            printf("Enter value: ");
            scanf("%d", &value);

            insertEnd(&head, value);
            break;

        case 3:
            printf("Enter value: ");
            scanf("%d", &value);

            printf("Enter position: ");
            scanf("%d", &position);

            insertPosition(&head, value, position);
            break;

        case 4:
            deleteBeginning(&head);
            break;

        case 5:
            deleteEnd(&head);

```

```

        break;

    case 6:
        printf("Enter position: ");
        scanf("%d", &position);

        deletePosition(&head, position);
        break;

    case 7:
        display(head);
        break;

    case 8:
        printf("Program terminated.\n");
        break;

    default:
        printf("Invalid choice! Please try again.\n");
    }

} while (choice != 8);

```

Sample Output

```

===== SINGLY LINKED LIST =====
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display / Traversal
8. Exit
=====
Enter your choice: 1
Enter value: 10
10 inserted at beginning.

Enter your choice: 2
Enter value: 20
20 inserted at end.

Enter your choice: 2
Enter value: 30
30 inserted at end.

Enter your choice: 7
Linked List: 10 -> 20 -> 30 -> NULL

Enter your choice: 3

```

```
Enter value: 15
Enter position: 2
15 inserted at position 2.

Enter your choice: 7
Linked List: 10 -> 15 -> 20 -> 30 -> NULL

Enter your choice: 6
Enter position: 3
20 deleted from position 3.

Enter your choice: 7
Linked List: 10 -> 15 -> 30 -> NULL

Enter your choice: 4
10 deleted from beginning.

Enter your choice: 5
30 deleted from end.
```

Complexity Analysis

Operation	Time Complexity
Insert at beginning	O(1)
Insert at end	O(n)
Insert at position	O(n)
Delete from beginning	O(1)
Delete from end	O(n)
Delete from position	O(n)
Traversal	O(n)

Space Complexity: $O(n)$ for storing n linked-list nodes.

Lab Task

Write, compile, and execute the above menu-driven program. Demonstrate at least one example of each insertion, deletion, and traversal operation. Record the algorithm, C program, input, and output in your laboratory notebook.